Programming and Numerical Methods for Economics (ECNM10115)

University of Edinburgh

Final Examination — Academic Year 2026/27

— Model Solutions —

Date: December 2026
Duration: 2 hours
Total marks: 50
Equipment: Word processor only (no Python environment)

Instructions.

- Answer **all** questions. Type your answers in the word processor provided.
- When asked to write code, type well-formatted Python code. You will be assessed on correctness of logic and syntax; you do *not* need to run the code.
- When asked for explanations, write concise prose (2–4 sentences unless stated otherwise).
- You may assume that `numpy`, `pandas`, `scipy`, and `matplotlib` are imported as usual:

```
import numpy as np
import pandas as pd
from scipy.optimize import fsolve, minimize
import matplotlib.pyplot as plt
```

Question 1 — Programming Fundamentals and Debugging [7 marks]

Part (a) [3 marks]

The following function is intended to simulate the **Solow growth model** for T periods starting from an initial capital stock k_0. The law of motion is

$$k_{t+1} \;=\; s\,A\,k_t^{\alpha} + (1-\delta)\,k_t\,.$$

The function should return the capital path $\{k_t\}_{t=0}^{T}$ and the output path $\{y_t\}_{t=0}^{T}$ where $y_t = A\,k_t^{\alpha}$.

The code below contains **three errors**. For each error, (i) state the line number, (ii) explain what is wrong, and (iii) write the corrected line.

```
1 def solow_simulate(k0, T, alpha, s, delta, A):
2     k = np.zeros(T)
3     k[0] = k0
4     for t in range(T):
5         k[t+1] = s * A * k[t]**alpha + delta * k[t]
6     y = A * k**A
7     return k, y
```

Solution

Error 1 — Line 2: `np.zeros(T)` allocates only T elements (indices $0, \ldots, T-1$), but we need $T+1$ values for $k_0, k_1, \ldots, k_T$. When $t = T-1$ the assignment `k[t+1] = k[T]` is out of bounds.
Corrected line 2: `k = np.zeros(T + 1)`

Error 2 — Line 5: The depreciation term uses `delta * k[t]` instead of `(1 - delta) * k[t]`. The law of motion requires the *undepreciated* fraction $(1-\delta)\,k_t$.
Corrected line 5: `k[t+1] = s * A * k[t]**alpha + (1 - delta) * k[t]`

Error 3 — Line 6: The exponent is `A` (the TFP parameter) instead of `alpha`. Output is $y_t = A\,k_t^{\alpha}$, so the exponent must be α.
Corrected line 6: `y = A * k**alpha`

Part (b) [4 marks]

Write a Python function `ces_production(K, L, A, alpha, sigma)` that computes output from the CES production function:

$$Y \;=\; A\Big(\alpha\, K^{\frac{\sigma-1}{\sigma}} + (1-\alpha)\, L^{\frac{\sigma-1}{\sigma}}\Big)^{\frac{\sigma}{\sigma-1}}.$$

Your function must handle the special case $\sigma = 1$, where the CES reduces to the **Cobb–Douglas** form $Y = A\,K^{\alpha}\,L^{1-\alpha}$. Include a brief docstring.

Solution

```python
def ces_production(K, L, A, alpha, sigma):
    """Compute output from a CES production function.
    Falls back to Cobb-Douglas when sigma = 1.
    """
    if sigma == 1:
        return A * K**alpha * L**(1 - alpha)
    else:
        rho = (sigma - 1) / sigma
        return A * (alpha * K**rho + (1 - alpha) * L**rho)**(1 / rho)
```

Key points: Define $\rho = (\sigma-1)/\sigma$ so the inner exponent is ρ and the outer exponent is $1/\rho = \sigma/(\sigma-1)$. The `if` guard is necessary because $\rho \to 0$ as $\sigma \to 1$, which causes division by zero.

Question 2 — Stochastic Processes and Simulation [9 marks]

Part (a) — AR(1) processes [4 marks]

An AR(1) process is $y_t = \rho\, y_{t-1} + \varepsilon_t, \quad \varepsilon_t \sim N(0, \sigma^2), \quad y_0 = 0.$

(i) Write a Python function `simulate_ar1(rho, sigma, T)` that returns a NumPy array of $T+1$ values $(y_0, y_1, \ldots, y_T)$. Use `np.random.normal` to draw the shocks.
(ii) For $|\rho| < 1$, derive the unconditional (long-run) variance $\mathrm{Var}(y_\infty)$.
(iii) Write the theoretical autocorrelation at lag k: $\mathrm{Corr}(y_t, y_{t-k}) = ?$
(iv) Explain what happens when $\rho = 1$ and why this case is called a "unit root".

Solution

(i)

```
def simulate_ar1(rho, sigma, T):
    y = np.zeros(T + 1)
    eps = np.random.normal(0, sigma, T)
    for t in range(T):
        y[t + 1] = rho * y[t] + eps[t]
    return y
```

(ii) In the stationary regime, $\mathrm{Var}(y_t) = \mathrm{Var}(y_{t-1}) \equiv V$. Taking variance of both sides (ε_t is independent of y_{t-1}):

$$V = \rho^2 V + \sigma^2 \quad \Longrightarrow \quad V(1-\rho^2) = \sigma^2 \quad \Longrightarrow \quad \boxed{\mathrm{Var}(y_\infty) = \frac{\sigma^2}{1-\rho^2}}.$$

(iii) $\mathrm{Corr}(y_t,\, y_{t-k}) = \rho^{|k|}$. (Autocorrelation decays geometrically with lag.)

(iv) When $\rho = 1$ the process becomes a *random walk*: $y_t = y_{t-1} + \varepsilon_t$. The variance grows without bound ($\mathrm{Var}(y_t) = t\,\sigma^2 \to \infty$), so no stationary distribution exists. It is called a "unit root" because the characteristic polynomial of the AR(1) is $z - \rho = 0$, whose root is $z = \rho = 1$ — i.e. the root lies exactly on the unit circle in the complex plane.

Part (b) — Markov chains [3 marks]

Consider a two-state Markov chain (Expansion, Recession) with transition matrix

$$P = \begin{pmatrix} 0.90 & 0.10 \\ 0.30 & 0.70 \end{pmatrix}.$$

GDP growth is $+2\%$ in Expansion and -1% in Recession.

(i) Compute the stationary distribution $\boldsymbol{\pi}$ analytically by solving $\boldsymbol{\pi} P = \boldsymbol{\pi}$ with $\pi_0 + \pi_1 = 1$. Show your working.
(ii) Using $\boldsymbol{\pi}$, compute the long-run average GDP growth rate.
(iii) Write a Python function `simulate_markov(P, T, s0)` that simulates T periods of the chain starting from state `s0`, using `np.random.choice` at each step. Return an integer array of states.

Solution

(i) From $\boldsymbol{\pi} P = \boldsymbol{\pi}$:

$$\pi_0 = 0.90\,\pi_0 + 0.30\,\pi_1$$
$$\pi_1 = 0.10\,\pi_0 + 0.70\,\pi_1$$

From the first equation: $0.10\,\pi_0 = 0.30\,\pi_1$, so $\pi_0 = 3\,\pi_1$. Substituting into $\pi_0 + \pi_1 = 1$:

$$3\pi_1 + \pi_1 = 1 \implies \pi_1 = 0.25, \quad \pi_0 = 0.75.$$

$$\boxed{\boldsymbol{\pi} = (0.75,\ 0.25)}$$

(ii) Long-run average GDP growth:

$$0.75 \times (+2\%) + 0.25 \times (-1\%) = 1.50\% - 0.25\% = \boxed{1.25\%}.$$

(iii)

```
def simulate_markov(P, T, s0):
    n_states = P.shape[0]
    states = np.zeros(T + 1, dtype=int)
    states[0] = s0
    for t in range(T):
        states[t + 1] = np.random.choice(n_states, p=P[states[t]])
    return states
```

Part (c) — Comparing persistence [2 marks]

Consider two AR(1) processes, both with $\sigma = 1$:

- Process A: $\rho = 0.5$
- Process B: $\rho = 0.95$

(i) Compute the long-run variance of each process. Which is more volatile in the long run and why?

(ii) The **half-life** of a shock is the number of periods until a unit shock decays to half its initial impact. Show that the half-life is $h = \ln 2 \,/\, \ln(1/|\rho|)$. Compute h for both processes.

Solution

(i) Using $\mathrm{Var}(y_\infty) = \sigma^2/(1-\rho^2)$ with $\sigma = 1$:
Process A: $V_A = 1/(1-0.25) = 1/0.75 = 1.333$.
Process B: $V_B = 1/(1-0.9025) = 1/0.0975 = 10.256$.
Process B is far more volatile in the long run — higher persistence amplifies the accumulated effect of shocks.

(ii) After a unit shock at time 0, the expected response at time h is ρ^h. Setting $\rho^h = 1/2$:

$$h \ln|\rho| = -\ln 2 \quad \Longrightarrow \quad h = \frac{\ln 2}{\ln(1/|\rho|)} = \frac{-\ln 2}{\ln|\rho|}.$$

Process A: $h = \ln 2 / \ln 2 = 1$ period.
Process B: $h = \ln 2 / \ln(1/0.95) = 0.6931/0.05129 \approx 13.5$ periods.

Question 3 — Monte Carlo Methods [8 marks]

Part (a) — Estimating π by simulation [3 marks]

A classic Monte Carlo exercise estimates π by generating random points in the unit square $[0,1]^2$ and counting how many fall inside the quarter circle of radius 1.

(i) Explain why $\pi/4 = P(X^2 + Y^2 \leq 1)$ when $X, Y \sim U(0,1)$ independently.
(ii) Write a Python function `estimate_pi(n)` that estimates π using n random points.
(iii) If $n = 10\,000$ and 7 856 points land inside the quarter circle, what is your estimate of π?

Solution

(i) The area of the quarter circle of radius 1 is $\pi \cdot 1^2/4 = \pi/4$. The area of the unit square is 1. Since (X, Y) is uniform over the square, the probability of falling inside the quarter circle equals the ratio of areas: $P(X^2 + Y^2 \leq 1) = \pi/4$.

(ii)

```
def estimate_pi(n):
    x = np.random.uniform(0, 1, n)
    y = np.random.uniform(0, 1, n)
    inside = np.sum(x**2 + y**2 <= 1)
    return 4 * inside / n
```

(iii) $\hat{\pi} = 4 \times 7856/10000 = 4 \times 0.7856 = 3.1424$.

Part (b) — Convergence of Monte Carlo estimators [2 marks]

Consider the Monte Carlo estimator $\hat{\theta}_n = \frac{1}{n}\sum_{i=1}^{n} g(X_i)$ for $\theta = E[g(X)]$ with $X_i \overset{\text{iid}}{\sim} N(0,1)$.

(i) For $g(x) = e^x$, what is the true value of θ? (*Hint*: the moment generating function of $N(0,1)$ is $M(t) = e^{t^2/2}$.)
(ii) Explain why the standard error of $\hat{\theta}_n$ decreases as $1/\sqrt{n}$. If you quadruple the number of draws, by what factor does the standard error decrease?

Solution

(i) $\theta = E[e^X] = M(1) = e^{1/2} = \sqrt{e} \approx 1.6487$.

(ii) By the CLT, $\hat{\theta}_n$ has variance $\text{Var}(g(X))/n$, so $\text{SE} = \sigma_g/\sqrt{n}$. The $1/\sqrt{n}$ rate comes from averaging n iid terms. Quadrupling n halves the SE: $1/\sqrt{4n} = \frac{1}{2} \cdot 1/\sqrt{n}$. This means gaining one extra decimal digit of precision requires roughly $100\times$ the draws.

Part (c) — Antithetic variates [3 marks]

The antithetic variates estimator for $\theta = E[e^X]$ with $X \sim N(0,1)$ replaces independent pairs (X_i, X_j) with correlated pairs $(X_i, -X_i)$:

$$\hat{\theta}_n^{\text{AV}} = \frac{1}{n}\sum_{i=1}^{n} \frac{e^{X_i} + e^{-X_i}}{2}.$$

(i) Explain in 2–3 sentences *why* this reduces variance. What property of the standard normal distribution does it exploit?
(ii) Write a Python function `mc_antithetic(n)` that returns the antithetic variates estimate of $E[e^X]$ using n draws.
(iii) State the Law of Large Numbers result that guarantees $\hat{\theta}_n^{\text{AV}} \to \theta$ as $n \to \infty$.

Solution

(i) The standard normal distribution is *symmetric* about zero, so X and $-X$ have the same distribution; thus e^{-X_i} is a valid draw from the same estimator. When X_i is large, e^{X_i} is large but e^{-X_i} is small, so the two terms are *negatively correlated*: $\text{Cov}(e^X, e^{-X}) < 0$. Averaging $\frac{1}{2}(e^{X_i} + e^{-X_i})$ exploits this negative covariance to reduce the variance compared to using two independent draws.

(ii)

```
def mc_antithetic(n):
    X = np.random.normal(0, 1, n)
    return np.mean((np.exp(X) + np.exp(-X)) / 2)
```

(iii) The **(Strong) Law of Large Numbers**: since $Z_i = \frac{1}{2}(e^{X_i} + e^{-X_i})$ are iid (each X_i is drawn independently) with $E[Z_i] = \theta$ and finite variance, the sample mean $\hat{\theta}_n^{\text{AV}} = \frac{1}{n}\sum Z_i$ converges almost surely to θ as $n \to \infty$.

Question 4 — Root-Finding and Fixed-Point Iteration [9 marks]

Part (a) — Newton's method and the IRR [3 marks]

Newton's method for $f(x) = 0$ uses the iteration $x_{n+1} = x_n - f(x_n)/f'(x_n)$.

(i) Write a Python function `newtons_method(f, fprime, x0, tol=1e-8, max_iter=100)` that implements Newton's method. It should return the root and the number of iterations. The loop should stop when $|f(x_n)| < \texttt{tol}$.

(ii) The Net Present Value of the cash-flow stream $C = (-1000, 300, 300, 300, 300, 300)$ is

$$\mathrm{NPV}(r) = \sum_{t=0}^{5} \frac{C_t}{(1+r)^t}\,.$$

Write the derivative $\mathrm{NPV}'(r)$ as an analytical formula. Then write the complete Python code (function definitions for NPV and NPV′, plus the call to your Newton solver) to find the Internal Rate of Return starting from $r_0 = 0.1$.

Solution

(i)

```
def newtons_method(f, fprime, x0, tol=1e-8, max_iter=100):
    x = x0
    for i in range(1, max_iter + 1):
        fx = f(x)
        if abs(fx) < tol:
            return x, i
        x = x - fx / fprime(x)
    return x, max_iter
```

(ii) Differentiating term by term:

$$\mathrm{NPV}'(r) = \sum_{t=1}^{5} \frac{-t\,C_t}{(1+r)^{t+1}}\,.$$

(The $t = 0$ term is constant and vanishes.)

```
C = [-1000, 300, 300, 300, 300, 300]

def npv(r):
    return sum(C[t] / (1 + r)**t for t in range(6))

def npv_prime(r):
    return sum(-t * C[t] / (1 + r)**(t + 1) for t in range(1, 6))

irr, n_iter = newtons_method(npv, npv_prime, 0.1)
# irr is approximately 0.1524 (15.24%)
```

Part (b) — Comparative statics with `fsolve` [3 marks]

The Solow steady state satisfies $s\,k^{\alpha} = (n+\delta)\,k$. Use $s = 0.3$, $\alpha = 0.33$, $n = 0.02$, $\delta = 0.05$.

(i) Define the residual function $g(k) = s\,k^{\alpha} - (n+\delta)\,k$ and write a one-line call to `fsolve` to find k^*.

(ii) Derive the closed-form solution $k^* = \left(\frac{s}{n+\delta}\right)^{1/(1-\alpha)}$. Compute k^* and $y^* = (k^*)^{\alpha}$ for the given parameters (show the numerical calculation).

(iii) Write Python code that varies s from 0.1 to 0.5 (using `np.linspace`) and stores the corresponding k^* for each s.

Solution

(i)

```
def g(k, s=0.3, alpha=0.33, n=0.02, delta=0.05):
    return s * k**alpha - (n + delta) * k

k_star = fsolve(g, 1.0)[0]
```

(ii) At steady state $s\,k^{\alpha} = (n+\delta)\,k$. Dividing both sides by k:

$$s\,k^{\alpha-1} = n+\delta \implies k^{\alpha-1} = \frac{n+\delta}{s} \implies k^{*} = \left(\frac{s}{n+\delta}\right)^{\frac{1}{1-\alpha}}.$$

Numerically:

$$k^{*} = \left(\frac{0.3}{0.07}\right)^{1/0.67} = (4.2857)^{1.4925} \approx 8.78.$$

$$y^{*} = (k^{*})^{0.33} = (8.78)^{0.33} \approx 2.05.$$

Check: $s\,y^{*} = 0.3 \times 2.05 = 0.615$ and $(n+\delta)\,k^{*} = 0.07 \times 8.78 = 0.615$. ✓

(iii)

```
s_values = np.linspace(0.1, 0.5, 50)
alpha, n, delta = 0.33, 0.02, 0.05
k_stars = (s_values / (n + delta))**(1 / (1 - alpha))
```

Part (c) — Fixed-point iteration [3 marks]

Fixed-point iteration solves $x = g(x)$ via $x_{n+1} = g(x_n)$.

(i) State the Banach fixed-point theorem condition that guarantees convergence.
(ii) A cobweb model gives the iteration $P_{t+1} = g(P_t) = \frac{8-2P_t}{3}$. Find the fixed point P^{*} algebraically. Then compute $|g'(P^{*})|$ and determine whether the iteration converges.
(iii) In 2–3 sentences, explain the conceptual link between fixed-point iteration and **value function iteration** in dynamic programming.

Solution

(i) The Banach fixed-point (contraction mapping) theorem states: if g is a contraction on a closed subset of a complete metric space — that is, there exists $L \in [0,1)$ such that $|g(x)-g(y)| \le L\,|x-y|$ for all x, y in the domain (equivalently, $|g'(x)| \le L < 1$ in the differentiable case) — then g has a unique fixed point x^{*} and the iteration $x_{n+1} = g(x_n)$ converges to it from any starting point.

(ii) Setting $P^{*} = g(P^{*})$:

$$P^{*} = \frac{8-2P^{*}}{3} \implies 3P^{*} = 8-2P^{*} \implies 5P^{*} = 8 \implies \boxed{P^{*} = 1.6}.$$

The derivative is $g'(P) = -2/3$, so $|g'(P^{*})| = 2/3 < 1$. Since the contraction condition is satisfied, the iteration **converges**.

(iii) In dynamic programming, the Bellman equation $V = T(V)$ defines the value function V as a fixed point of the Bellman operator T. Value function iteration starts from an initial guess V_0 and iterates $V_{n+1} = T(V_n)$, which is exactly a fixed-point iteration in function space. Convergence is guaranteed because the discounting condition ($\beta < 1$) ensures T is a contraction mapping (by Blackwell's sufficient conditions).

Question 5 — Optimisation [9 marks]

Part (a) — Analytical utility maximisation [4 marks]

A consumer maximises Cobb–Douglas utility $U(x_1, x_2) = x_1^{\alpha}\, x_2^{1-\alpha}$ subject to the budget constraint $p_1 x_1 + p_2 x_2 = m$. Use $\alpha = 0.4$, $p_1 = 2$, $p_2 = 3$, $m = 120$.

(i) Set up the Lagrangian $\mathcal{L} = x_1^{\alpha}\, x_2^{1-\alpha} - \lambda(p_1 x_1 + p_2 x_2 - m)$ and derive the first-order conditions with respect to x_1, x_2, and λ.
(ii) Show that the Marshallian demand functions are $x_1^* = \alpha\, m/p_1$ and $x_2^* = (1-\alpha)\, m/p_2$.
(iii) Compute the optimal bundle $(x_1^*,\, x_2^*)$ for the given parameter values.

Solution

(i) The Lagrangian is $\mathcal{L} = x_1^{\alpha}\, x_2^{1-\alpha} - \lambda(p_1 x_1 + p_2 x_2 - m)$.
First-order conditions:

$$\frac{\partial \mathcal{L}}{\partial x_1} = \alpha\, x_1^{\alpha-1}\, x_2^{1-\alpha} - \lambda\, p_1 = 0 \tag{1}$$

$$\frac{\partial \mathcal{L}}{\partial x_2} = (1-\alpha)\, x_1^{\alpha}\, x_2^{-\alpha} - \lambda\, p_2 = 0 \tag{2}$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = -(p_1 x_1 + p_2 x_2 - m) = 0 \tag{3}$$

(ii) From (1): $\lambda = \frac{\alpha\, x_1^{\alpha-1}\, x_2^{1-\alpha}}{p_1}$. From (2): $\lambda = \frac{(1-\alpha)\, x_1^{\alpha}\, x_2^{-\alpha}}{p_2}$.
Setting equal and simplifying:

$$\frac{\alpha\, x_2}{p_1} = \frac{(1-\alpha)\, x_1}{p_2} \quad \Longrightarrow \quad p_2\, x_2 = \frac{(1-\alpha)}{\alpha}\, p_1\, x_1\,.$$

Substituting into the budget constraint (3):

$$p_1 x_1 + \frac{(1-\alpha)}{\alpha}\, p_1 x_1 = m \implies p_1 x_1 \cdot \frac{1}{\alpha} = m \implies \boxed{x_1^* = \frac{\alpha\, m}{p_1}}\,.$$

By symmetry (or substituting back): $\boxed{x_2^* = \frac{(1-\alpha)\, m}{p_2}}$.

(iii) Plugging in $\alpha = 0.4$, $p_1 = 2$, $p_2 = 3$, $m = 120$:

$$x_1^* = \frac{0.4 \times 120}{2} = 24, \qquad x_2^* = \frac{0.6 \times 120}{3} = 24.$$

Optimal bundle: $\boxed{(x_1^*,\, x_2^*) = (24,\, 24)}$.

Part (b) — Numerical optimisation with `scipy` [3 marks]

Write complete Python code using `scipy.optimize.minimize` with `method='SLSQP'` to solve the problem in Part (a) numerically. Your code must include the objective function (negated, since `minimize` minimises), the budget constraint as an equality constraint, bounds ensuring $x_1, x_2 > 0$, an initial guess, and the `minimize` call.

Solution

```
alpha = 0.4
p1, p2, m = 2, 3, 120

def neg_utility(x):
    return -(x[0]**alpha * x[1]**(1 - alpha))

def budget_constraint(x):
    return m - p1 * x[0] - p2 * x[1]    # = 0 at feasibility

x0 = [10.0, 10.0]
bounds = [(0.01, None), (0.01, None)]
constraints = {'type': 'eq', 'fun': budget_constraint}

result = minimize(neg_utility, x0, method='SLSQP',
                  bounds=bounds, constraints=constraints)
x_star = result.x   # approximately [24.0, 24.0]
```

Key points: We negate utility because `minimize` finds the minimum. The budget constraint is passed as an `'eq'` type (equality, meaning the function should equal zero). Bounds keep both goods strictly positive to avoid 0^{α} issues.

Part (c) — The Rosenbrock function [2 marks]

The Rosenbrock function is $f(x, y) = (1 - x)^2 + 100\,(y - x^2)^2$.

(i) What is the global minimum value and where does it occur?

(ii) Compare the **Nelder–Mead** and **BFGS** methods: which uses gradient information? Which would you expect to converge faster on the Rosenbrock function, and why?

Solution

(i) The global minimum is $f(1, 1) = 0$, occurring at $(x, y) = (1, 1)$. Both terms vanish there: $(1 - 1)^2 = 0$ and $(1 - 1^2)^2 = 0$.

(ii) BFGS uses gradient information — it is a quasi-Newton method that builds an approximate inverse Hessian from successive gradient evaluations. **Nelder–Mead** is derivative-free (simplex method). BFGS is expected to converge faster on the Rosenbrock function because the gradient and curvature information help it navigate the narrow, banana-shaped valley more efficiently, whereas Nelder–Mead can only contract its simplex slowly along the flat valley floor.

Question 6 — Neural Networks and the Euler Equation [8 marks]

This question considers the use of neural networks to solve dynamic economic models, following the *Deep Equilibrium Nets* approach applied to the Brock–Mirman (1972) optimal growth model.

A social planner maximises lifetime utility:

$$\max_{\{C_t\}_{t=0}^{\infty}} \sum_{t=0}^{\infty} \beta^t \ln(C_t) \qquad \text{s.t.} \quad K_{t+1} + C_t = K_t^{\alpha},$$

where we assume full depreciation ($\delta = 1$), so $Y_t = K_t^{\alpha}$ and $C_t = K_t^{\alpha} - K_{t+1}$.

A feedforward neural network $\mathcal{N}(\cdot)$ approximates the **savings rate** $s_t = \mathcal{N}(K_t)$, so that $K_{t+1} = Y_t \cdot s_t$.

Part (a) — Neural network architecture [3 marks]

The network has the following architecture:

- Input layer: 1 neuron (capital stock K_t)
- Hidden layer 1: 50 neurons, ReLU activation
- Hidden layer 2: 50 neurons, ReLU activation
- Output layer: 1 neuron, sigmoid activation

(i) Compute the total number of trainable parameters (weights and biases) in this network. Show the calculation for each layer.

(ii) Explain why the sigmoid activation function ($\sigma(z) = 1/(1+e^{-z}) \in (0,1)$) is used in the output layer. What economic constraint does this enforce on s_t, C_t, and K_{t+1}?

(iii) If the sigmoid were replaced by a linear (identity) activation in the output layer, what could go wrong when evaluating the loss function?

Solution

(i) Each dense layer has (inputs × outputs) weights plus (outputs) biases:

Connection	Weights	Biases	Total
Input → Hidden 1 (1×50)	50	50	100
Hidden 1 → Hidden 2 (50×50)	2500	50	2550
Hidden 2 → Output (50×1)	50	1	51
Grand total			**2701**

(ii) The sigmoid ensures $s_t \in (0,1)$, which enforces two economic constraints. First, $K_{t+1} = Y_t \cdot s_t > 0$, guaranteeing positive capital. Second, $C_t = Y_t(1 - s_t) > 0$, guaranteeing positive consumption. Both are necessary because the utility function $\ln(C_t)$ is undefined for $C_t \leq 0$.

(iii) With a linear output, the network could predict $s_t > 1$ (implying $C_t = Y_t(1 - s_t) < 0$, i.e. negative consumption) or $s_t < 0$ (implying negative investment). The log utility $\ln(C_t)$ would be undefined, and the loss function computation would fail with a numerical error (log of a negative number).

Part (b) — Deriving the Euler equation [3 marks]

The Bellman equation for this problem (with $\delta = 1$) is:

$$V(K_t) = \max_{K_{t+1}} \ln\big(K_t^{\alpha} - K_{t+1}\big) + \beta\, V(K_{t+1}).$$

(i) Take the first-order condition with respect to K_{t+1}. Then apply the envelope theorem to express $V'(K_t)$ in terms of C_t and K_t, and use this to eliminate $V'(K_{t+1})$ from the FOC.

Show that the Euler equation is:

$$\frac{1}{C_t} = \beta \frac{\alpha K_{t+1}^{\alpha-1}}{C_{t+1}}.$$

(ii) Rewrite this Euler equation as a **relative Euler equation error** (REE) that equals zero in equilibrium:

$$\text{REE} = \frac{C_{t+1}}{C_t \beta \alpha K_{t+1}^{\alpha-1}} - 1.$$

Explain in one sentence why mean(REE^2) is a suitable training loss for the neural network.

Solution

(i) The FOC with respect to K_{t+1}:

$$\frac{\partial}{\partial K_{t+1}}\Big[\ln\big(K_t^\alpha - K_{t+1}\big) + \beta V(K_{t+1})\Big] = 0 \quad \Longrightarrow \quad -\frac{1}{C_t} + \beta V'(K_{t+1}) = 0. \qquad (*)$$

By the envelope theorem, differentiate $V(K_t)$ w.r.t. K_t (noting K_{t+1} is optimal and its derivative vanishes by FOC):

$$V'(K_t) = \frac{\alpha K_t^{\alpha-1}}{K_t^\alpha - K_{t+1}} = \frac{\alpha K_t^{\alpha-1}}{C_t}.$$

Shifting one period forward: $V'(K_{t+1}) = \frac{\alpha K_{t+1}^{\alpha-1}}{C_{t+1}}$.
Substituting back into $(*)$:

$$\frac{1}{C_t} = \beta \frac{\alpha K_{t+1}^{\alpha-1}}{C_{t+1}}.$$

(ii) Rearranging the Euler equation:

$$1 = \frac{C_{t+1}}{C_t \beta \alpha K_{t+1}^{\alpha-1}} \quad \Longrightarrow \quad \text{REE} = \frac{C_{t+1}}{C_t \beta \alpha K_{t+1}^{\alpha-1}} - 1.$$

The mean squared REE is a suitable loss because it measures how far the neural network's policy deviates from optimality: when the policy is exactly optimal, the Euler equation holds for all states and REE $= 0$ everywhere, so the loss is zero; during training, gradient descent drives the loss toward zero, pushing the neural network policy toward the true optimal policy.

Part (c) — Interpreting the training code [2 marks]

The following function computes the training loss for the neural network. Assume `nn(K)` returns the predicted savings rate $s = \mathcal{N}(K)$.

```
1  def compute_cost(K_t, nn, alpha=0.36, beta=0.99):
2      Y_t = K_t ** alpha                          # Line 2
3      s_t = nn(K_t)                               # Line 3
4      K_next = Y_t * s_t                          # Line 4
5      C_t = Y_t - Y_t * s_t                       # Line 5
6      Y_next = K_next ** alpha                    # Line 6
7      s_next = nn(K_next)                          # Line 7
8      C_next = Y_next - s_next * Y_next           # Line 8
9      R_next = alpha * K_next ** (alpha - 1)      # Line 9
10     REE = 1 - C_next / (beta * C_t * R_next)    # Line 10
11     return np.mean(REE ** 2)                     # Line 11
```

(i) The neural network `nn` is called *twice* (Lines 3 and 7). Explain the economic reason: what role does each call play in the Euler equation?

(ii) If the neural network has perfectly learned the true optimal policy, what value should `compute_cost` return? What does the returned value represent when the network is *not* yet fully trained?

Solution

(i) The Euler equation links consumption in *two consecutive periods*, C_t and C_{t+1}. The first call (Line 3) computes today's savings rate $s_t = \mathcal{N}(K_t)$, which determines today's consumption C_t and tomorrow's capital K_{t+1}. The second call (Line 7) computes tomorrow's savings rate $s_{t+1} = \mathcal{N}(K_{t+1})$, which determines tomorrow's consumption C_{t+1}. Both are needed because the Euler equation $1/C_t = \beta\,\alpha K_{t+1}^{\alpha-1}/C_{t+1}$ involves the policy at two different states.

(ii) If the neural network has learned the true optimal policy, the Euler equation holds exactly for all states, so REE $= 0$ everywhere and `compute_cost` returns $\boxed{0}$. When the network is not yet fully trained, the returned value is the **mean squared relative Euler equation error** — a positive number measuring how far, on average, the current policy violates the optimality condition. A smaller value means the approximation is closer to the true solution.

End of Examination